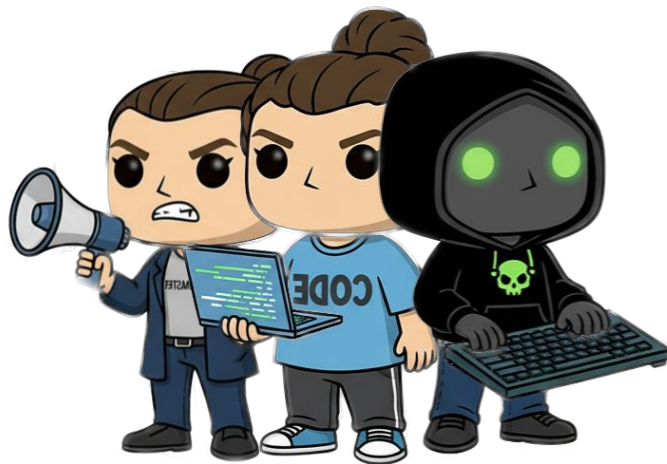


The Big Software Supply Chain Security Problem

Jozsef Ottucsak

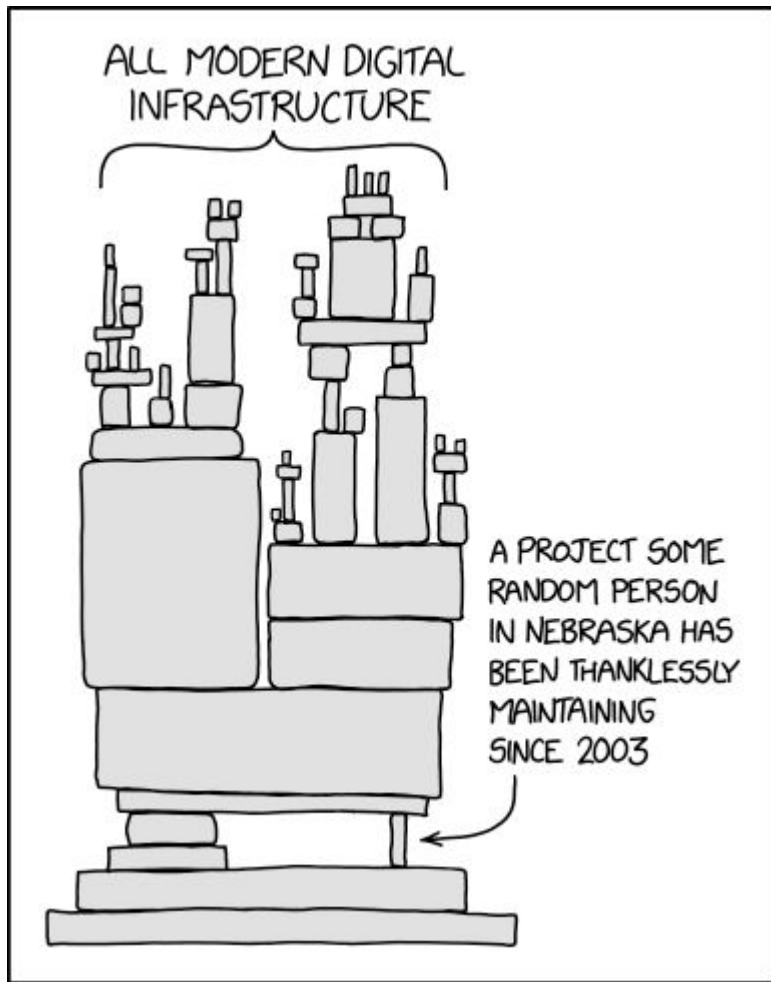
Agenda

- Introduction
- Why software supply chain matters
- How does software supply chain work
- Frameworks, Controls, Complications



Who am I?

- Jozsef Ottucsak (@fuzboxz) - Product Security Architect
- Enthusiastic about helping people make the right decisions in security
- 10+ years in cybersecurity
- CISSP, OSCP, ISO27001, CCSKv4, MSC, MBA
- Loves technology, especially high tech-low life aspect
 - Currently tinkering with adapting zero trust for my local network
 - Loves DIY and open-source electronic music instruments
 - Was into AI before it was cool



What's up with Supply Chain Security?

Why are you here? - Part 1

- npm left-pad (2016)
 - Developer removed left-pad package from npm and thousands of applications broke because they relied on it. The OG supply chain security warning.
- SolarWinds (2020)
 - Nation state compromised build system and backdoored IT management software binaries which affected approximately 18,000 customers including US federal agencies and major tech firms..
- Log4j (2021)
 - Widely used logging library vulnerable to unauthenticated RCE due to poorly secured message substitution feature. Affected hundreds of millions of devices and 93% of cloud environments.
- MOVEit Transfer (2023)
 - Russian ransomware group exploited SQL injection in managed file transfer software. 2700 companies and almost a 100 million individuals affected directly or indirectly..

Why are you here? - Part 2

- XZ Utils Backdoor (2024)
 - Malicious backdoor was introduced into xz compression binary that allowed the attacker remote access through SSH. Core utility used in most Linux distributions.
- Polyfill.io (2024)
 - Chinese company acquired Polyfill.io domain name and Github account. Shortly after they started redirecting users to malicious sites and deployed malware. Over 1.6 million hosts referenced the service.
- Shai-Hulud (2025)
 - Targeted phishing compromised package maintainers, after getting access the attackers deployed a self-replicating worm that harvested credentials and replicated itself by injecting it into packages maintained by the victim and publishing compromised versions to the registry.

State of the Industry

- Modern systems rely on thousands of small moving parts directly or indirectly. One weak link could compromise the whole chain.
- Dependencies are high value targets, compromising one frequently used package can grant access to thousands or millions of systems.
- Lot of critical projects are maintained by a single person or small team with no security budget.
- **Regulators, customers and partners expect integrity, transparency and secure handling of 3rd party dependencies and open-source code.**

EO 14028 - Improving the Nation's Cybersecurity (2021)

Established foundational baseline for software supply chain security

- Software Bill of Materials (SBOM) required for transparency
- Secure software development practices by NIST standards
- Stronger information sharing regarding incidents and vendor breaches
- Higher expectations for "critical software" in federal environments

EO 14144 – Strengthening Cybersecurity Innovation (2025)

Focus on operational enforcement of requirements and practices

- Vendor attestations and artifacts (SBOM, test results, secure SDLC proof)
- NIST SP 800-161 Supply Chain Risk Management guidelines enforced
- Federal acquisition contracts require secure software practices
- Open source security including guidance, patching and ecosystem contributions

EU Cyber Resilience Act

Applicable to products with digital elements - except for open-source, software with no local components (SaaS/PaaS) and heavily regulated industries.

- Required cyber risk assessment
- Secure by design and default development processes
- **Supply chain due diligence**
- Vulnerability handling and remediation
- Incident and vulnerability reporting
- Evidence: Technical documentation and **SBOM**

(Not a full list, please refer to other resources)

Software Supply Chain Ecosystem

WE DON'T WANT TO REINVENT THE WHEEL,
SO EVERY DAY WE GOOGLE IMAGE SEARCH
"WHEEL," AND WHATEVER OBJECT COMES UP,
THAT'S WHAT WE ATTACH TO OUR VEHICLES.

SURE, EXTERNAL DEPENDENCIES
CARRY RISKS, BUT SO FAR THEY'VE
ALL BEEN PRETTY GOOD WHEELS.



Overview

Before we can secure anything we have to understand:

- What is the complete scope of our software supply chain?
- What are the components that go into the final artifact?
- Where do we source the components that are used?
- How do we distribute/host our applications?

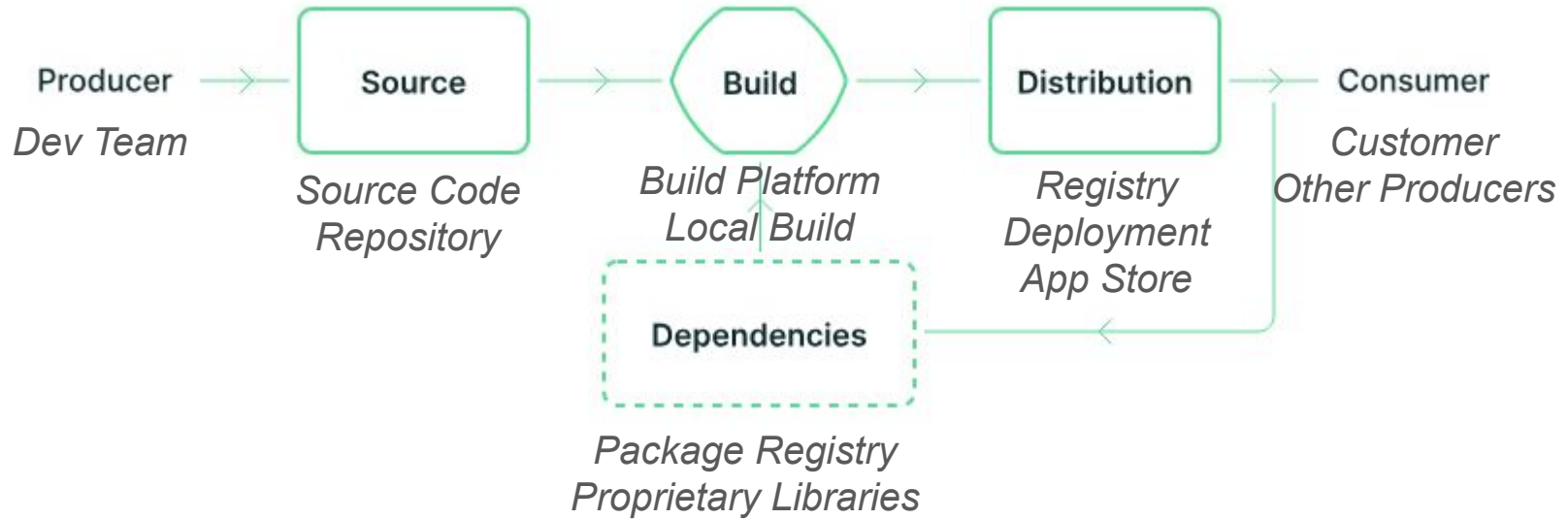
Application Components

Software isn't just code, it's a stack of components.

- First-Party Code (written by us)
- Third-Party Code (written by others)
- Documentation, know-how (docs)
- Build & Delivery Pipeline (runtimes, compilers, manifests)
- Configuration & Environments (runtime configs, secrets)
- Infrastructure & Platforms (OS/hypervisor, images, cloud platforms)

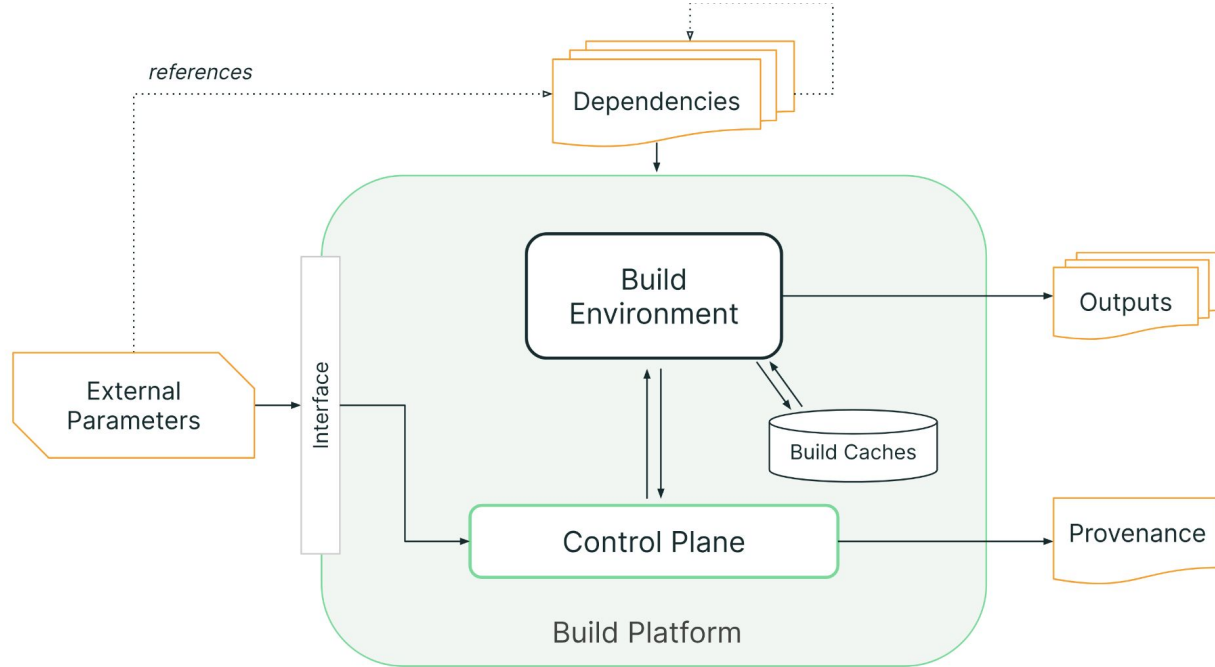
How and where do we ensure trust with all these moving parts?

Software Supply Chain



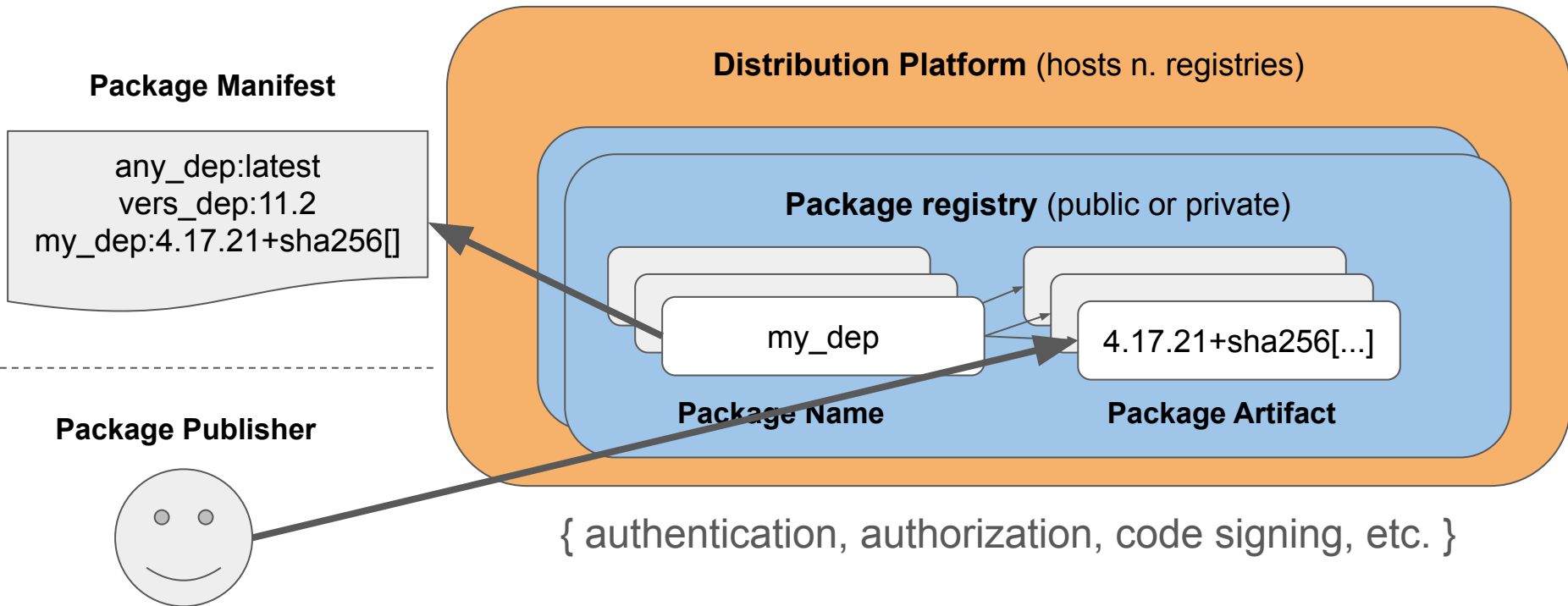
Source: SLSA

Build Model



Source: SLSA

Distribution Model



Key Threat Categories

- Source Code Threats
 - Malicious code injected into codebase
- Build Environment Threats
 - Build process compromised to inject malware
- Dependency Threats
 - Compromised or vulnerable 3rd party components
- Deployment Threats
 - Deploying compromised/malicious artifacts

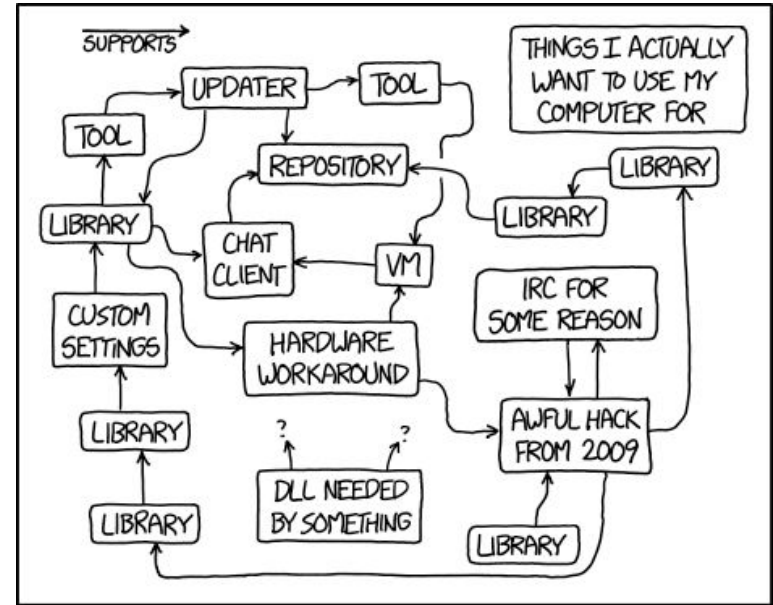


There is a framework for that!

Multiple security frameworks exist that address the problem from different angles:

- OWASP Software Component Verification Standard
- OpenSSF SLSA (Supply-chain Levels for Software Artifacts)
- NIST 800-218 Secure Software Development Framework
- NIST 800-161 Cybersecurity Supply Chain Risk Management (C-SCRM)
- OpenSSF Security Score Cards (for open-source)
- Cloud Native Computing Foundation in-toto
- Multiple relevant ISO/IEC standards (5230, 27036, 20243)

Software Supply Chain Strategy



EVERY NOW AND THEN I REALIZE I'M MAINTAINING A
HUGE CHAIN OF TECHNOLOGY SOLELY TO SUPPORT ITSELF.

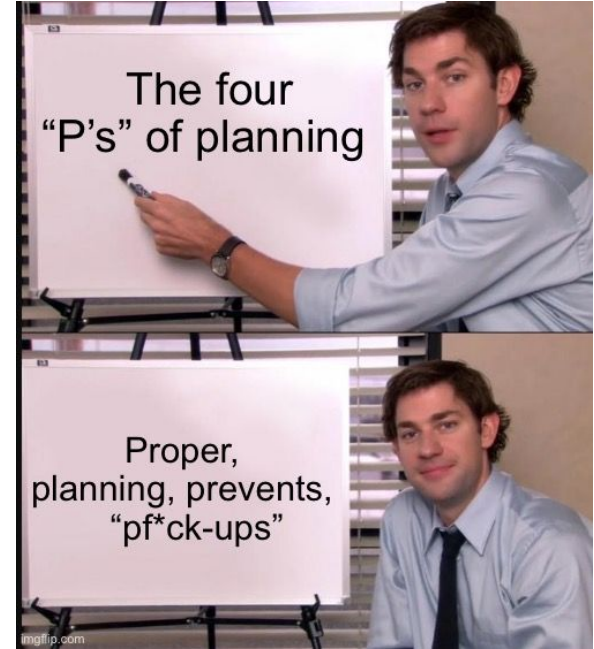
Why we need supply chain security

- **CTO:** That NPM worm was wild. What can we do to prevent issues like this?
- **Sales:** I can't close this deal unless you provide an up-to-date SBOM to the customer.
- **Legal:** Non-compliance with EU CRA can lead to 2.5% annual turnover fines.
- **Developer:** No idea if we are affected by the latest big vulnerability, we have a dozen of repositories and I'm only responsible for a few of them.
- **Customer:** Is 1.5.76a version of your software affected by the latest big vulnerability?
- **Cyber Insurance:** Your premiums will increase if you can't demonstrate that you have software supply chain security protections in place.

You Need A Plan

- External Requirements
 - Requirements often already specified
 - Define roadmap and execute
- Internal Drivers
 - Identify risks related to software supply chain
 - Map risks to strategic initiatives
 - Prioritize the mitigation of biggest risks
 - Define roadmap and execute

Adopt a supply chain risk management process!



Supply Chain Security Maturity Models

OWASP SCVS, OpenSSF SLSA provides guidance on levels of assurance

- Incremental controls ranging from fundamental to advanced
- Could be used as frameworks, benchmark or just inspiration
- Pick level (or controls) based on your security requirements
- SCVS closer on broader software supply chain security controls
- SLSA focuses on automated software provenance
- There is overlap, but the two are complementary
- For most companies SCVS is the right choice

Secure Software Development Lifecycle

Hopefully you already have this in place, but a quick refresher

- Threat Modeling
- Code Reviews
- Software Composition Analysis
- Static Application Security Testing
- Dynamic Application Security Testing
- Security Baselines - Hardening
- Penetration Testing



Supply chain security deals with third-party risk, not the security of internal code

Open Source/3rd Party Library Policy

Adopt some sort of policy to control the use of 3rd party components

- Acceptable version drift from latest version (even if non-vulnerable)
- Criteria for selection (popularity, contributors, last update, etc.)
- End-of-life/end-of-support policy
- Supplier SBOM requirements
- License policy (open-source)
- SLA for remediation

Let's Talk
About
Technical
Controls!



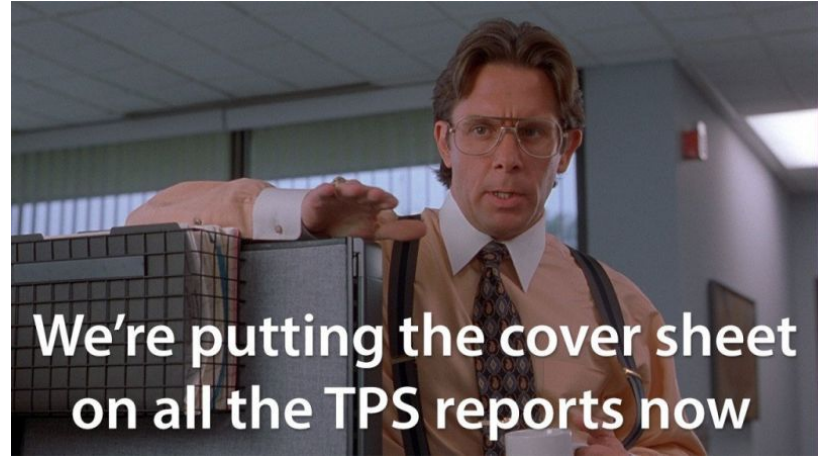
SBOM

"You can't secure what you can't see"

- Software Bill of Materials (SBOM) is the logical place to start
- Provides visibility into what components go into your software both for you and your customers
- Machine-readable, can be enriched with vulnerability data
- Can be used to validate exposure to vulnerabilities or license violations
- Quick win, easy to implement (Github SBOM API or build step)
- Mandated by EU Cyber Resilience Act from December 2027

CISA Required SBOM Fields as of 2025

- Software Producer
- SBOM Author
- Generation Context
- Tool Name
- Timestamp
- *Per Component*
 - *Name*
 - *Version*
 - *Unique ID*
 - *Hash*
 - *License*
 - *Dependency Relationship*



SBOM Complications - Part 1

- Naive implementations could be noisy and incorrect
 - Pre-build (manifest) / post-build (container inspection) based approaches
 - Unused, QA and dev dependencies might be included in SBOM
 - Build scripts or non-standard workflows might be ignored
 - Might not include all required CISA SBOM parameters
- How to deal with different package managers and formats?
 - CycloneDX - favored by security community
 - SPDX - popular in the open-source community
 - No one-size-fits-all tooling, coverage gaps even with commercial solutions
 - Not every tool up-to-date with CISA requirements

SBOM Complications - Part 2

- How to deal with monorepos?
 - Incompatible with naive implementations (bloated root SBOM)
 - Version conflicts possible between projects
 - Might include dead code, obsolete, R&D or utility projects
 - Generate SBOM per project during- or post-build
- How to package everything and distribute it?
 - Ideally SBOM should be packaged with the build artifact
 - An application might contain multiple artifacts (merge or zip?)
 - SBOM contains sensitive information about the entire software stack
 - Manual - email/CRM (pull) vs automated publishing - API (push)

Securing and Hardening the CI Pipeline - Part 1

- Isolated & Ephemeral Builds
 - Each build should run in clean, temporary environment (containers)
 - Prevents cross-build attacks or establishing persistence on server
 - Common in the cloud native world
- Infrastructure-as-Code CI/CD Pipeline
 - Build infrastructure defined and managed as code
 - Version control, branch protection, multi-approval PRs
 - All the changes are controlled and tracked
 - Protection against compromised administrators, configuration drift

Securing and Hardening the CI Pipeline - Part 2

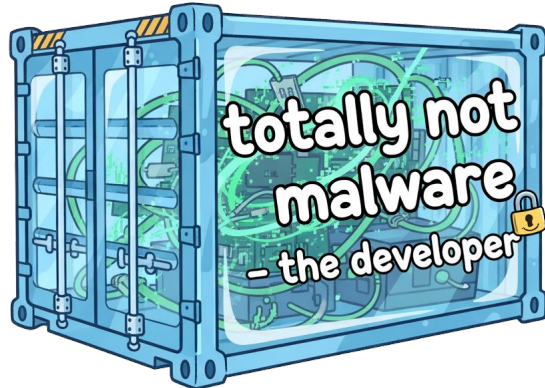
- Deterministic Builds
 - Build process always produces the exact same output based on the inputs
 - Pinning **all external parameters** prevents external influence on build
 - Bit-by-bit reproduction means that independent builders can verify integrity
- Hermetic Builds
 - Strict network isolation for build environment
 - Can only access trusted internal registries
 - Prevents attacker from data exfiltration or fetching unauthorized code
 - Requires centralized artifact repository
 - Requires pinned inputs from *Deterministic Builds*

CI/CD Pipeline Complications

- Isolated & Ephemeral Builds - IaaS CI/CD Pipeline
 - Increased complexity
 - Steeper learning curve compared to traditional operations
- Deterministic Builds
 - Even smallest inconsistencies prevent reproducible builds
 - Source and build definitions are tightly coupled
 - Hard to gather all of the parameters influencing the build
- Hermetic Builds
 - Dependency management overhead
 - Developer friction - no ad-hoc experimentation

Artifact Signing

- Validate source identity and integrity
- Can be verified at consumption (build or deployment)
- Does not ensure that the code or its dependencies are not malicious
- Requires key management and automated signing process
- Upstream third-party and open-source dependencies are often not signed



Build Provenance

- Signed attestation about how the software was built
- Signature and policy verification
 - Source code and build commit SHA
 - Build source (pipeline ID)
 - Dependencies
 - Verification steps (tests, security tooling)
- Can be useful during investigation as well
- Adopted by security-sensitive fields like cloud, defense, fintech, open source
- Requires secure, isolated and ephemeral builds to make sense

Trusted Publishers

- Uses short lived OIDC based authentication for CI jobs
- Eliminates long lived API keys for publishing
- Reduces the risk of credential theft
- Identity is bound to the CI job
- Verifiable audit trail of source in case of compromise

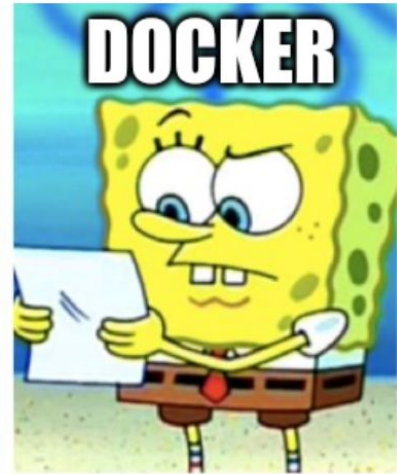
Not universally supported by all CI/CD pipelines



Rebuilding Third Party Components

- Components are rebuilt in-house
- Full security analysis based on internal policies
- Vulnerabilities reported, possibly fixed and committed back to source
- Requires constant effort and dedicated resources
- Used where high level of security assurance is required
- Common in big tech and cloud providers

tl;dr



Summary

- If you produce software expect more scrutiny from customers
- Gather the SBOMs from your software suppliers
- Constantly evolving area, changing requirements
- Plenty of things to do, up to you to choose the right tasks
- Some things will be hard to do on certain platforms/stacks
- Every control comes at a cost (money, complexity, time, office politics)
- Perfect is the enemy of good - something is better than nothing

Thank you! :)

maybe time for questions?